

EEE 402 (July 2025)

Artificial Intelligence and Machine Learning Laboratory

Final Project Report

Section: G2 Group: 03

Camera-to-camera Style Transfer

Course Instructors:

Dr. Mohammad Ariful Haque, Professor
Akif Hamid, Part-Time Lecturer

Signature of Instructor: _____

Academic Honesty Statement:

IMPORTANT! Please carefully read and sign the Academic Honesty Statement, below. Type the student ID and name, and put your signature. You will not receive credit for this project experiment unless this statement is signed in the presence of your lab instructor.

"In signing this statement, We hereby certify that the work on this project is our own and that we have not copied the work of any other students (past or present), and cited all relevant sources while completing this project. We understand that if we fail to honor this agreement, We will each receive a score of ZERO for this project and be subject to failure of this course."

Signature: _____

Full Name: Roisul Islam Dibyo

Student ID: 2006125

Signature: _____

Full Name: Arif Zakaria

Student ID: 2006126

Signature: _____

Full Name: Md. Fahim Rahman

Student ID: 2006128

Signature: _____

Full Name: Md. Marziul Haque Riad

Student ID: 2006129

Table of Contents

1	Abstract.....	1
2	Introduction	1
3	Design	1
3.1	Problem Formulation (PO(b)).....	1
3.1.1	Identification of Scope	1
3.1.2	Literature Review.....	2
3.1.1	Formulation of Problem.....	2
3.1.2	Analysis	3
3.2	Proposed Method (PO(a)).....	3
3.2.1	Model Components.....	3
3.2.2	Loss Functions	5
3.2.3	Dataset	6
4	Implementation	7
4.1	Model Hyperparameters.....	7
4.2	Dataset and Dataloader	8
4.3	Training.....	9
4.4	Notebook & GitHub Link	9
5	Results	10
5.1	Style Transfer Across Training Epochs	10
5.2	Style Diversity Across Camera Domains.....	10
5.3	Style Consistency in the Same Domain:	11
5.4	Loss vs Epochs.....	12
5.5	Analysis of Style Representations	12
6	Reflection on Individual and Team work (PO(i))	13
6.1	Individual Contribution of Each Member.....	13
6.2	Mode of TeamWork.....	14
7	Future Work (PO(l)).....	14
8	References	15

1 Abstract

In this project, we present a novel application of StarGAN v2 for camera model style transfer, where the objective is to convert the visual style of an image from one camera model (e.g., iPhone-6) to another (e.g., Sony-NEX-7) while preserving its core content. Unlike conventional domain transfer methods that rely on latent noise vectors, our approach utilizes only the Style Encoder from StarGAN v2, enabling purely reference-guided translation. We use the SP-Society Camera Model Identification dataset, comprising ten distinct camera classes, and preprocess it into high-variance 256×256 image patches to ensure rich visual information. To maintain data balance, we apply stratified splitting and construct a custom dataset class that performs appropriate augmentations. A suite of DataLoaders feeds both source and reference images into the model. During training, we modify the original StarGAN v2 framework by removing the Mapping Network and adapting the loss functions to support style transfer solely through encoded references. We employ adversarial, style consistency, cycle consistency, and diversity-sensitive losses, with gradual decay on diversity regularization. Our results demonstrate visually compelling style transfer across camera domains, validated by qualitative grid visualizations of source-reference-generated triplets. The system shows promise in fooling camera model classifiers, facilitating research into forensic robustness, and offers a pathway to exploring style interpolation and hybridization across imaging devices. This work serves as a step forward in understanding and manipulating the intrinsic visual signatures of different camera models using deep generative models.

2 Introduction

In the realm of digital imagery, every camera tells its own story—not through words, but through pixels. From the soft contrast of an iPhone lens to the vivid saturation of a Sony sensor, each device embeds its unique stylistic fingerprint into the images it captures. These stylistic differences, often imperceptible to the untrained eye, can have a profound impact on downstream applications such as forensic analysis, media consistency, and synthetic data generation.

This project aims to decode and reimagine those stylistic signatures through the power of deep generative models. Leveraging a customized StarGAN v2 architecture, we embark on a journey to perform camera model-specific style transfer: re-rendering images in the visual language of different camera domains while preserving their semantic content. By removing the latent noise pathway and focusing solely on reference-guided style injection, our approach offers granular control over the stylistic transformation—enabling one-to-one stylization based on specific reference images.

Built upon the SP-Society Camera Model Identification dataset, our pipeline not only captures the essence of domain adaptation but also offers insights into the variability of style within the same camera model. With applications spanning digital forensics to creative content generation, this work illustrates how modern machine learning can blend perception and aesthetics—translating the language of lenses into a canvas of computational style

3 Design

3.1 Problem Formulation (PO(b))

3.1.1 Identification of Scope

Modern image datasets often suffer from inconsistencies introduced by varying camera devices, which embed distinct visual characteristics such as color tones, contrast levels, and sharpness. These stylistic differences can adversely affect the performance of image-based machine learning tasks such as object detection, classification, and forensic analysis. The scope of this project is to develop a system that

can accurately extract and transfer camera-specific visual styles from one image to another while preserving the content, using deep learning techniques. This enables augmentation of datasets, domain adaptation, and style normalization across varying devices.

3.1.2 Literature Review

The advancement of image style transfer, particularly in the context of camera-aware image-to-image translation, has been strongly influenced by generative adversarial networks (GANs). A considerable body of research has explored how camera-induced visual variations affect downstream vision tasks such as person re-identification, image generation, and denoising. These studies form the foundation for our approach to camera-specific style transfer.

Traditional approaches often sought to make feature extractors invariant to camera domains, relying on hand-engineered representations or metric learning techniques. However, more recent works have shifted toward explicit modeling of camera styles using generative models. One notable example is the use of CycleGAN for camera-style transfer, which enabled the synthesis of images from one camera's domain to another, enhancing domain generalization through data augmentation. However, CycleGAN-based methods typically require separate models for each domain pair and often suffer from noisy translations due to the absence of domain supervision.

To address this, newer frameworks like StarGAN and its variants (e.g., SP-StarGAN) have emerged, enabling multi-domain image-to-image translation using a single unified model [2][4]. These methods introduced structural and identity-preserving losses to ensure that content remains unchanged while the style is adapted. They demonstrated that enforcing consistency across different camera styles can improve the quality of generated samples, especially in challenging tasks like person re-identification [1][2].

Another parallel line of work focused on real-world sensor artifacts and noise. In domains like denoising, GANs have been employed not just to clean images but also to learn realistic noise distributions stemming from camera hardware [3]. This underscores the importance of modeling camera-specific characteristics directly from data—an idea closely related to our objective of camera model-based style transformation.

Informed by these developments, our project leverages the style-aware generative capacity of StarGAN v2 [4], but with architectural modifications for pure reference-guided stylization. By removing latent noise injection and relying solely on StyleEncoder-based conditioning, we are able to model subtle style differences between cameras without requiring domain-level averaging or artificial supervision. Our design decisions were directly shaped by the insights and limitations identified in the prior art. The general concept of disentangled style representation is also influenced by the design of StyleGAN, which demonstrated effective style-based control in image synthesis [5].

3.1.1 Formulation of Problem

Given a source image x_s taken from an unknown camera domain D_s , and a reference image x_r from a target camera domain D_t , develop a model F such that:

$$F(x_s, x_r) \rightarrow x'$$

where x' retains the content of x_s but adopts the style of x_r .

The challenge lies in extracting a consistent style representation from x_r that is sufficiently informative to guide the stylization of x_s while preserving semantic fidelity. Furthermore, the model must ensure diversity within the same domain and maintain generalization across domains.

Constraints:

- Model must support per-image stylization.

- No use of latent noise; purely reference-based style extraction.
- Training must ensure cycle consistency, adversarial realism, and style accuracy.

3.1.2 Analysis

Through detailed exploration and testing, we observed that camera styles are not just domain-level labels but exhibit high intra-domain variance. For instance, images taken by the same iPhone model can still differ due to lighting, exposure, and minor lens inconsistencies. This justifies the need for reference-guided style transfer as opposed to domain-to-domain translation.

Our analysis included:

- Measuring L2 norm differences in style vectors between images within the same and across different camera domains.
- Visual inspection of generated outputs across multiple source-reference pairs.
- Evaluating stability with/without diversity-sensitive loss (λ_{ds}). While training λ_{ds} was gradually annealed to zero to ensure stability.

This analytical grounding guided our architectural simplifications and helped establish a robust, interpretable, and flexible style transfer pipeline tailored for camera-specific tasks.

3.2 Proposed Method (PO(a))

To achieve reference-guided camera style transfer across multiple domains, we build on **StarGAN v2** — a generative adversarial framework designed for multi-domain and diverse image-to-image translation. StarGAN v2 addresses two key challenges in image translation:

1. **Generating diverse outputs** for a given source image, and
2. **Scaling to multiple target domains** using a single model.

Earlier models often addressed only one of these aspects. Some approaches generated diverse outputs but were limited to two domains, while others handled multiple domains but produced deterministic results with no variation. StarGAN v2 overcomes this by introducing a unified model that supports both diversity and multi-domain scalability, making it well-suited for camera-aware style transfer tasks.

At the core of StarGAN v2 is the idea of style-based generation, where the target style is encoded not by a simple label but by a style vector, either sampled from a latent distribution or extracted from a reference image. This enables fine-grained control over the output appearance and supports reference-guided stylization — a key requirement for our project.

StarGAN v2 introduces two modules for obtaining style codes:

1. A **Mapping Network**, which transforms random noise into a style vector for a given domain
2. A **Style Encoder**, which extracts a style vector directly from a reference image

In our project, we focus exclusively on the **reference-guided path** using the **Style Encoder**, bypassing the Mapping Network and latent sampling. This choice allows us to preserve the per-image characteristics of each camera model by conditioning generation directly on real style examples.

3.2.1 Model Components

Our proposed model is built upon the StarGAN v2 framework, customized for the task of camera-aware image-to-image translation. The original StarGAN v2 supports multi-domain and diverse image synthesis using both random latent sampling and reference-based style extraction. In our implementation, we remove the Mapping Network and diversity loss, relying solely on reference-guided stylization via the Style Encoder. This decision reflects the nature of our task: camera models

have subtle but consistent visual signatures, and our focus is on accurately transferring those rather than generating arbitrary stylistic variation.

The architecture consists of three primary modules — the Generator, the Style Encoder, and the Discriminator — adapted as follows:

1. Generator:

The Generator performs the core image translation. It receives a source image and a style code (extracted from a reference image) and synthesizes an output image in the style of the target camera. The Generator follows an encoder-decoder architecture:

- Downsampling blocks reduce spatial dimensions while capturing semantic features from the source image.
- Residual blocks perform transformations on these compact representations.
- Upsampling blocks reconstruct the image, using Adaptive Instance Normalization (AdaIN) to inject style information from the reference.

The AdaIN layers adjust the feature maps using scaling and shifting parameters derived from the style code. This mechanism enables precise control over visual attributes such as color, contrast, and texture.

2. Style Encoder (E):

The Style Encoder extracts the camera-specific style code from a reference image. This module is essential for reference-guided translation, where stylization is grounded in real visual patterns rather than random noise.

The Style Encoder is composed of:

- Shared convolutional layers that extract base-level visual features
- Domain-specific output heads (one per camera model) that project these features into a compact style vector

This style vector encodes the visual fingerprint of the reference camera, capturing attributes such as:

- Color Profile: temperature, saturation, and color response curves
- Sensor Noise: subtle grain or low-light noise signatures
- Contrast and Tonality: how shadows and highlights are rendered
- Sharpness and Texture: camera-specific edge enhancement
- Lens Artifacts: vignetting, blur patterns, etc.

By removing the Mapping Network — which in the original StarGAN v2 learns to generate random style vectors from latent noise — we ensure that all stylizations in our model are grounded in actual reference data.

3. Discriminator (D):

The Discriminator serves its standard adversarial role, distinguishing between real and generated images, and verifying whether generated outputs belong to the correct camera domain. It is structured as a **multi-task CNN** with domain-specific output heads. Each branch classifies whether an input image is a real or fake instance of a specific camera model. This multi-branch design improves stability and precision during training.

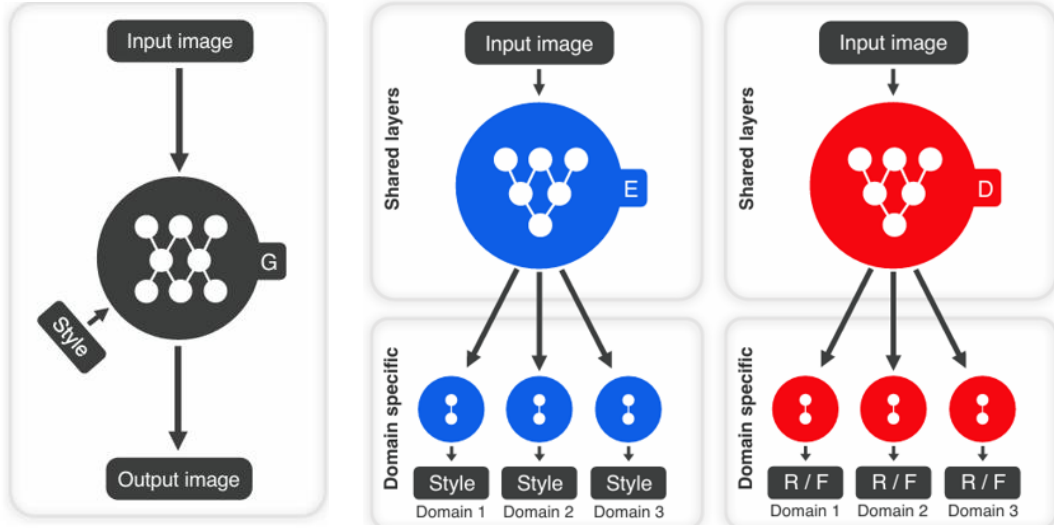


Figure 1: Components of our Proposed Model (From StarGAN v2)

3.2.2 Loss Functions

The training of StarGAN v2 is guided by four distinct loss functions, each with a specific purpose. These work together to ensure the model produces images that are not only high-quality but also diverse and faithful to both the source content and the target style.

1. Adversarial Loss (L_{adv})

This is the fundamental GAN loss that pushes the generator to create realistic images. It's a competitive game between the Generator (G), which tries to create convincing fakes, and the Discriminator (D), which tries to get better at spotting them.

$$\mathcal{L}_{adv} = \mathbb{E}_{x,y} [\log D_y(x)] + \mathbb{E}_{x,\tilde{y},z} [\log(1 - D_{\tilde{y}}(G(x, \tilde{s})))]$$

The first term is for real images. The Discriminator D tries to maximize this by making its output $D_y(x)$ (the probability that a real image x is real) as close to 1 as possible.

The second term is for fake images. The Generator G creates a fake image $G(x, \tilde{s})$. The Discriminator tries to make $D_{\tilde{y}}(G(x, \tilde{s}))$ close to 0 (identifying it as fake), which maximizes this term. The Generator, in turn, tries to make this value close to 1 (fooling the Discriminator), which minimizes this part of the loss.

2. Style Reconstruction Loss (L_{sty})

This loss ensures that the Generator actually uses the provided style code $\tilde{s}$ when creating an image.

$$\mathcal{L}_{sty} = \mathbb{E}_{x,\tilde{y},z} [\| \tilde{s} - E_{\tilde{y}}(G(x, \tilde{s})) \|_1]$$

Mathematical Analysis:

- $\tilde{s} = F_y(z)$: The target style code generated by the Mapping Network F from a random latent code z .
- $G(x, \tilde{s})$: The output image generated using the style code $\tilde{s}$.
- $E_{\tilde{y}}(G(x, \tilde{s}))$: The style code that the Style Encoder E extracts from the newly generated image.
- **The L1 norm, $\|\cdot\|_1$** , calculates the difference between the original style code and the one extracted from the output. By minimizing this loss, the model forces the Generator G to produce an image that accurately reflects the intended style $\tilde{s}$.

3. Style Diversification Loss ($\mathcal{L}_{ds}$)

This loss explicitly encourages the Generator to produce a wide variety of images for a given input. Equation:

$$\mathcal{L}_{ds} = \mathbb{E}_{x, \tilde{y}, z_1, z_2} \left[\|G(x, \tilde{s}_1) - G(x, \tilde{s}_2)\|_1 \right]$$

Mathematical Analysis:

- $\tilde{s}_1$ and $\tilde{s}_2$: Two different style codes generated by the Mapping Network F from two different random latent codes, z_1 and z_2 .
- $G(x, \tilde{s}_1)$ and $G(x, \tilde{s}_2)$: The two different images generated from the same input x but with different style codes.
- The L1 norm measures the pixel-wise difference between the two generated images. The objective is to maximize this distance, which forces the Generator to learn meaningful style features that result in visually distinct outputs.

4. Cycle Consistency Loss ($\mathcal{L}_{cyc}$)

This loss ensures that the Generator preserves the original, domain-invariant characteristics of the input image (like pose or identity) while changing its style.

Equation:

$$\mathcal{L}_{cyc} = \mathbb{E}_{x, y, \tilde{y}, z} \left[\|x - G(G(x, \tilde{s}), \hat{s})\|_1 \right]$$

Mathematical Analysis:

- $G(x, \tilde{s})$: The input image x is first translated into a target domain $\tilde{y}$.
- $\hat{s} = E_y(x)$: The Style Encoder E extracts the *original* style code from the input image x .
- $G(G(x, \tilde{s}), \hat{s})$: The translated image is then translated back to the original domain y using its original style code $\hat{s}$.

The L1 norm measures the difference between the original input x and the "cycled" image. By minimizing this loss, the model ensures that the content of the image remains consistent after the round-trip translation.

Thus, the full objective function can be written as,

$$\min_{G, F, E} \max_D \mathcal{L}_{adv} + \lambda_{sty} \mathcal{L}_{sty} - \lambda_{ds} \mathcal{L}_{ds} + \lambda_{cyc} \mathcal{L}_{cyc}$$

3.2.3 Dataset

The dataset used in this project is based on the **IEEE SP Cup 2018 Camera Model Identification Challenge**, available via Kaggle. It comprises high-resolution photographs taken by 10 different consumer camera models, including devices like the iPhone-6, LG Nexus 5x, Motorola Droid MAXX, Samsung Galaxy Note3, Sony NEX-7, and others.

To prepare the data for training the style transfer model, each image was processed to extract multiple **256×256 patches** using high-variance cropping, ensuring that the resulting samples contain meaningful texture rather than blank or low-detail areas. These patches were organized in a structured directory format such as `train/{camera_model}/*.png`. For inference, the data was further split into two components: a `src_img/` folder containing source images to be stylized, and a `ref_img/{camera_model}/` directory where each subfolder contains reference images representing a specific camera's style. Camera model names were encoded using `LabelEncoder` to serve as domain labels for the StarGAN v2 model's multi-domain style encoder.

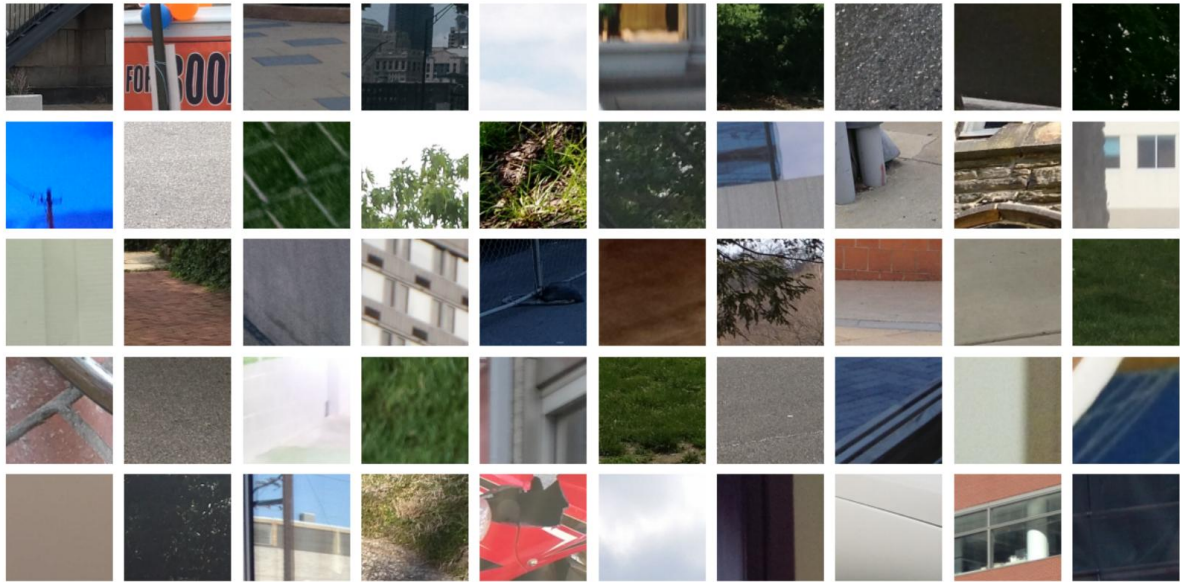


Figure 2: Sample images from the dataset

4 Implementation

We used the official PyTorch implementation of StarGANv2 which can be found at: <https://github.com/clovaai/stargan-v2>

4.1 Model Hyperparameters

```
nf = 64      # Number of base filters in Generator/Discriminator
nd = 10      # Number of camera model domains (classes)
sdim = 64    # Dimension of the style vector (style code output from StyleEncoder)
```

- These parameters define the model architecture size and the number of domains (camera types).
- The style dimension `sdim` determines how richly the style of a camera can be encoded.

```
lambda_gp = 1.  # Gradient penalty for discriminator stability
lambda_sty = 1. # Style consistency loss to match style vectors
lambda_ds = 1.  # Diversity-sensitive loss to ensure varied outputs
lambda_cyc = 1. # Cycle consistency loss to preserve content
```

- These scalar weights balance the contribution of different loss components.
- They are crucial for stabilizing adversarial training and ensuring high-quality, diverse style transfer results.

```
lr = 1e-4      # Learning rate for G, D, and E
betas = (0.0, 0.99) # Adam optimizer beta parameters
weight_decay = 1e-4 # Regularization to prevent overfitting
```

- These define the behavior of the **Adam optimizer**, which is used for training all three networks (G, D, E).
- Proper tuning of learning rate and regularization is essential to avoid instability in GAN training.

```
batch_size = 4      # Number of image samples per training step
epochs = 30         # Total training duration
```

```
ds_epochs = 5          # Annealing period for diversity-sensitive loss
n_print = 100          # Logging frequency (every 100 iterations)
n_img_save = 2000      # Frequency of saving visual results
```

- These parameters control the **flow of training**—how long, how often logs and images are saved, and how batch-wise updates are handled.
- The `ds_epochs` setting linearly reduces the weight of diversity loss after 5 epochs to prioritize stability.

4.2 Dataset and Dataloader

```
img_path = glob('/kaggle/input/ieee-sp-cup-camera-model-identification-dataset/*/*')
print(f"Image Loaded: {len(img_path)}")
```

- Uses `glob` to recursively fetch all image file paths from the dataset directory.
- Each subfolder represents a camera model class, and the total number of images is printed (55,000 in this case).

```
df = pd.DataFrame(img_path, columns=['image_path'])
df['camera_model'] = df['image_path'].apply(lambda x: x.split('/')[2])
```

- Converts the list of image paths into a Pandas DataFrame for easier manipulation.
- Extracts the camera model name from the directory structure using `.split('/')[2]`

```
le = LabelEncoder()
df['camera_model'] = le.fit_transform(df['camera_model'])
```

- A `LabelEncoder` from `sklearn.preprocessing` is used to convert string-based camera model names (e.g., "iPhone-6") into integer labels (e.g., 0, 1, ..., 9).
- The transformation ensures compatibility with PyTorch-based models, which operate on integer class labels during both training and loss computation.

The output looks like:

```
camera_model
6    5500
8    5500
9    5500
5    5500
0    5500
2    5500
7    5500
3    5500
4    5500
1    5500
```

```
train_df, test_df = train_test_split(
    df,
    test_size=0.1,          # Reserve 10% of data for testing
    stratify=df['camera_model'], # Preserve class balance in split
    random_state=42         # For reproducibility
)
```

- **`train_test_split`** from `sklearn.model_selection` is used to split the dataset.
- The **`stratify`** parameter ensures that each camera model is equally represented in both training

and test sets—essential for preventing evaluation bias.

- **random_state=42** makes the split reproducible, so results are consistent across runs.

```
train_dataset = CameraPatchDataset(train_df['image_path'], train_df['camera_model'],
transform='train')

test_dataset = CameraPatchDataset(test_df['image_path'], test_df['camera_model'],
transform='test')

dataloader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True, num_workers=2,
pin_memory=True)
ref_dataloader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True,
num_workers=2, pin_memory=True)
#ref2_dataloader = DataLoader(train_dataset, ...)
test_dataloader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False,
num_workers=2, pin_memory=True)
ref_test_dataloader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False,
num_workers=2, pin_memory=True)
```

We used a custom CameraPatchDataset class to construct PyTorch-compatible training and testing datasets. These datasets applied mode-specific image transformations (augmentation for training, normalization for testing). We then configured five DataLoader objects:

- dataloader: provides source images and their labels for training.
- ref_dataloader: supplies reference images to guide style injection.
- test_dataloader: used for generating predictions on unseen test images.
- ref_test_dataloader: provides style references for test-time transfer evaluation.

All loaders were designed for efficient GPU training using mini-batches, with multi-threading (num_workers=2) and pinned memory for optimal performance

4.3 Training

We trained our modified StarGAN v2 model for **28 epochs**, using exclusively **reference-guided style transfer** without the mapping network. Each epoch involved feeding the generator with a source image and a reference image, extracting a domain-specific style vector via the Style Encoder, and applying it to generate stylized outputs. The training incorporated adversarial loss, style reconstruction loss, cycle consistency loss, and a diversity-sensitive loss, with `lambda_ds` linearly annealed to zero across epochs to stabilize generation. Each epoch took approximately **3 hours** on a single GPU, resulting in a total training time of around **84 hours**. Snapshots of generated images, along with model checkpoints and loss logs, were saved after each epoch to monitor progression and fine-tune hyperparameters as needed.

4.4 Notebook & GitHub Link

Here are the relevant links:

1. The Kaggle notebook
<https://www.kaggle.com/code/arifzakaria126/cam-to-cam-style-transfer>
2. GitHub Repository
<https://github.com/Arif-Nirzon/Camera-to-Camera-Style-Transfer>

5 Results

5.1 Style Transfer Across Training Epochs

The figure shows how the model improves reference-guided style transfer over training epochs. In early stages (Epoch-5, Epoch-10), the generated images appear noisy and poorly stylized. As training progresses, the model produces more realistic and style-consistent outputs while preserving the content. By Epoch-25 and Epoch-28, the style from iPhone-6 and GalaxyNote3 is clearly reflected, indicating effective learning and domain adaptations



Fig 3. Generated images across epochs

5.2 Style Diversity Across Camera Domains

The generated images exhibit noticeable differences depending on the reference camera style, indicating successful style transfer. For the same input content, variations such as color tone, contrast, sharpness, and illumination can be observed across domains like **iPhone-6**, **GalaxyNote3**, **Sony-NEX-7**, and **Motorola-Nexus-6**. For example, **iPhone-style outputs** tend to appear cooler and less saturated, while **Motorola and HTC styles** produce warmer tones or higher contrast. These domain-specific characteristics show that the model learns and applies diverse style features from different camera distributions, while still preserving the structural content of the original image.

Reference Image

Generated Images for various cameras

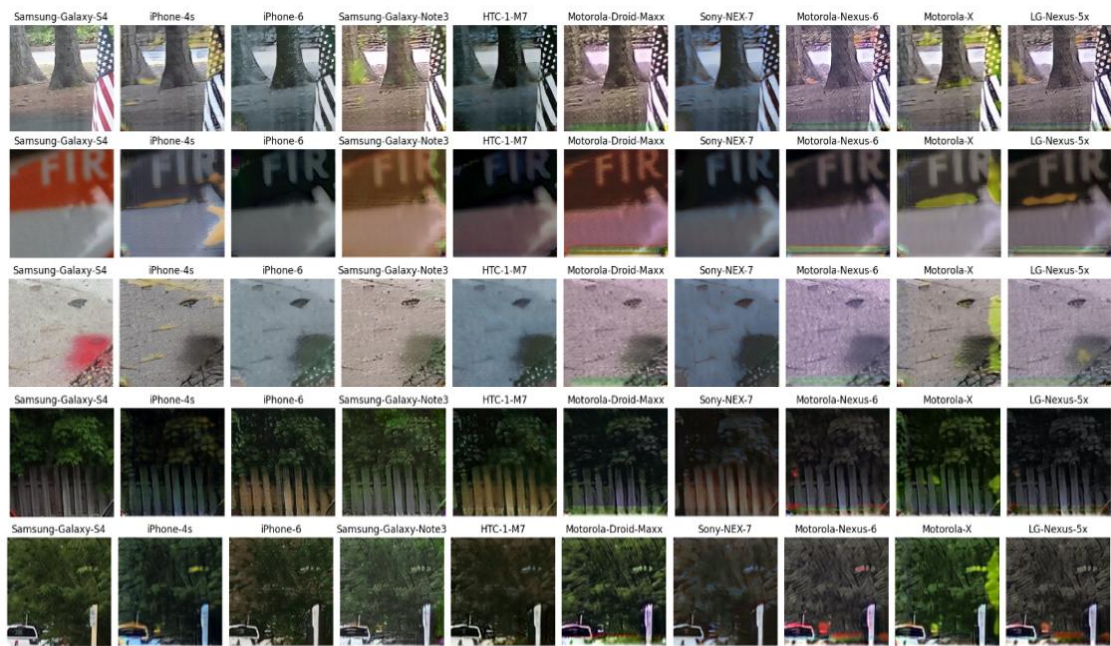


Fig 3. Single source to many camera style transfer

5.3 Style Consistency in the Same Domain:

The generated images for **iPhone 6** and **Samsung Galaxy S4** closely resemble the visual characteristics of real images from their respective cameras. The model effectively captures and transfers camera-specific style elements such as color tone, contrast, and texture. This visual consistency indicates successful domain learning and highlights the model's capability to mimic authentic camera styles.

Generated Images of iPhone6



Original Images of iPhone6



Generated Images of Samsung Galaxy S4



Original Images of Samsung Galaxy S4



Fig 4. Style Consistency of the same cameras

5.4 Loss vs Epochs

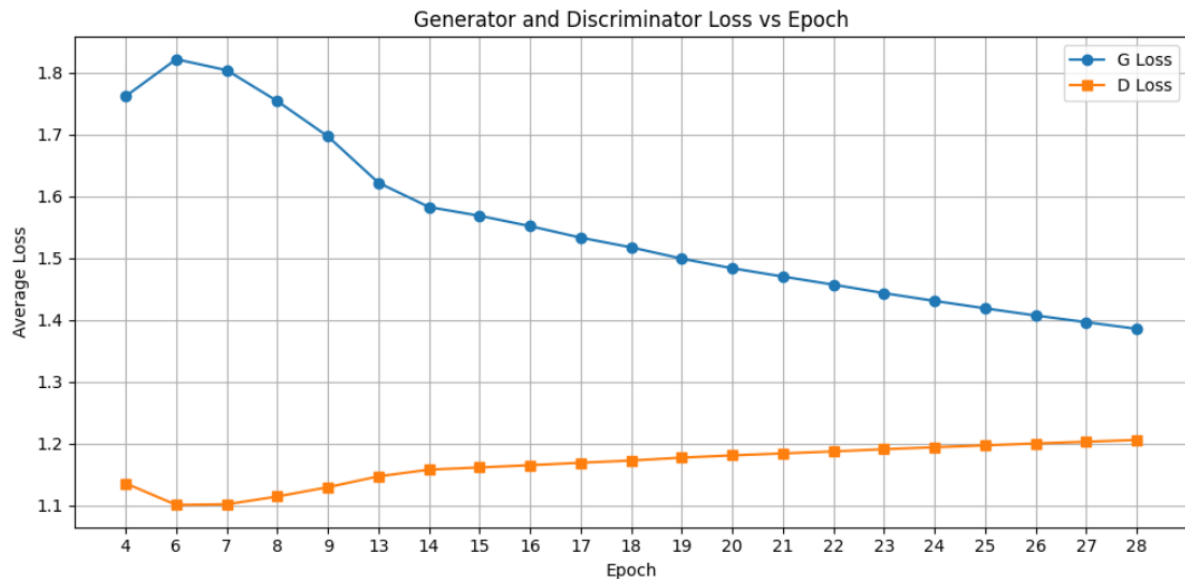


Fig 5. Generated and Discriminator Loss

The graph shows that the **generator loss (G Loss)** consistently decreases over epochs, indicating that the generator is improving in producing more realistic images. Conversely, the **discriminator loss (D Loss)** slightly increases over time, which is expected as the discriminator finds it increasingly difficult to distinguish between real and generated images. This opposing trend suggests that the adversarial training is stable and the generator is successfully learning to fool the discriminator.

5.5 Analysis of Style Representations

To better understand how the Style Encoder learns to capture camera-specific visual traits, we performed t-SNE (t-distributed Stochastic Neighbor Embedding) on the style vectors extracted from reference images. For each image, we passed it through the trained Style Encoder $E(x, y)$ using its corresponding camera label y , and collected the resulting style vector. The 2D t-SNE projection of

these vectors reveals clear clustering patterns by camera model, demonstrating that the encoder successfully learns to separate domains in its latent space. Interestingly, even within the same camera class, we observe intra-class variance, reflecting subtle stylistic differences due to lighting, texture, or content — which aligns with our observation that style vectors vary across images.

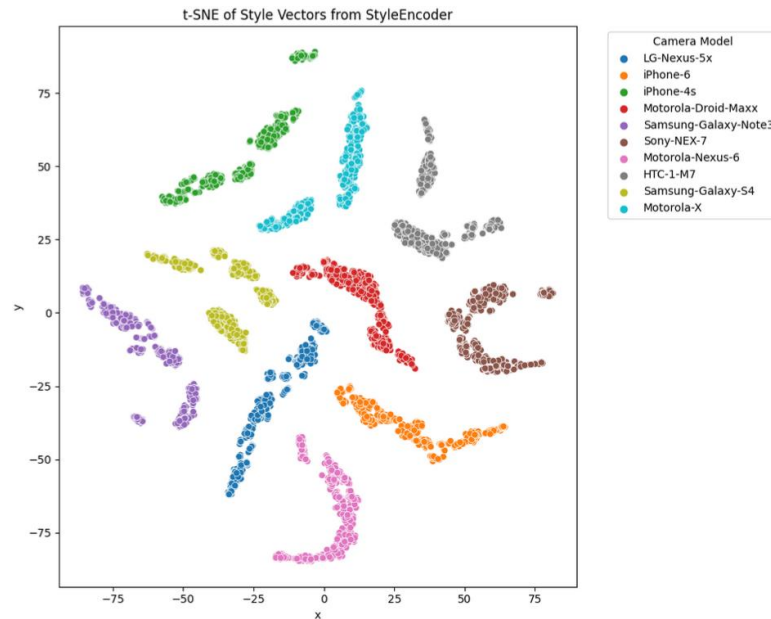


Fig 6. Reduced Dimensionality Visualization

6 Reflection on Individual and Team work (PO(i))

6.1 Individual Contribution of Each Member

Member	Responsibilities
2006125	Conducted literature review on GANs, StarGAN v2, and camera style transfer - Managed dataset preprocessing: patch extraction, label encoding, and stratified train-test split - Summarized related work and previous methodologies, Report Writing.
2006126	Implemented model architecture: ResBlock, AdaIN, AdaINResBlock - Built and integrated Generator, StyleEncoder, and Discriminator - Designed Model wrapper for saving/loading, optimizers, and debugging, Report Writing.
2006128	Designed and implemented loss functions: adversarial, style, cycle, and diversity-sensitive - Modified StarGAN v2 to remove Mapping Network (reference-only approach) - Configured training loop and hyperparameter tuning, Report Writing.
2006129	Developed inference pipeline using $G(x, E(x_{ref}))$ - Created visualizations (Source / Generated / Reference grids) - Performed complexity analysis: model size, GPU usage, and speed - Interpreted and summarized results, Report Writing.

6.2 Mode of TeamWork

Date	Milestone achieved
26-04-2025	Proposal making
08-05-2025	Literature review and complexity analysis.
12-05-2025	Dataset finding complexity and discussing probable methods.
13-05-2025	Project update
01-06-2025	Dataset pre-processing and model Implementation.
15-06-2025	Model Training started
20-07-2025	Training Ended
25-07-2025	Inference, Result analysis
28-07-2025	Report Writing

7 Future Work (PO(I))

1. Using a Pre-trained Camera Identification Model as the Style Encoder:

A model already trained to identify cameras has, by definition, learned to extract the *exact features* that differentiate one camera from another. These features (noise patterns, color profiles, etc.) are precisely what you define as "style." By using this pre-trained model as our style encoder, the generator will get a massive head start with a highly specialized "style expert. we could take a model like ResNet, EfficientNet, or another architecture that performed well in the IEEE SP Cup challenge, freeze its initial layers (which have learned the crucial features), and train only the final few layers to output the 64-dimensional style code. This would likely lead to a much more robust and accurate style extraction.

2. Implementing New Loss Functions:

The standard L1 loss for cycle consistency is good at preserving content, but it can sometimes struggle with textures and fine details. Adding more sophisticated loss functions can significantly improve the perceptual quality of the generated images.

- **Perceptual Loss (LPIPS):** Instead of comparing raw pixel values, this loss compares images in a deep feature space (using a network like VGG or AlexNet). This often aligns much better with how humans perceive similarity and can help generate more realistic textures.
- **Feature Matching Loss:** We could add a loss that encourages the statistics (like mean and variance) of feature maps in the discriminator to match between real and generated images. This can help the generator produce more realistic styles at different scales.
- **Identity Loss:** To better preserve the original image's content, we can add a loss that minimizes the feature-space distance between the input image and the generated image. This complements the cycle-consistency loss and can prevent the generator from making

unnecessary changes to the scene.

3. Training for More Epochs:

This is a straightforward but effective step. More training time allows the model to learn more subtle details and fine-tune its results.

4. Higher Resolution Training:

Many subtle camera artifacts, like fine sensor noise and lens aberrations, are more visible at higher resolutions. Training your model on 512x512 or even 1024x1024 images could produce much more convincing and detailed results. This would likely require modifying the generator to have more upsampling/downsampling blocks.

5. Quantitative Evaluation of Style Transfer:

We could develop an objective test. Train a separate, simple camera identification classifier. Then, feed it our generated images and see how often it correctly identifies their *intended* camera style. A high classification accuracy would be strong evidence that our style transfer is successful.

8 References

- [1] Y. Choi, Y. Uh, J. Yoo, and J.-W. Ha, "StarGAN v2: Diverse Image Synthesis for Multiple Domains," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2020, pp. 8188–8197. <https://arxiv.org/abs/1912.01865>
- [2] H.-Y. Chung, Y. Lee, and J. Yoo, "Camera-Aware Image-to-Image Translation Using Similarity Preserving," in *European Conf. Computer Vision (ECCV)*, 2022. <https://arxiv.org/abs/2204.05693>
- [3] K. Tran, Y. Dai, and L. Liu, "GAN-Based Noise Model for Denoising Real Images," in *Asian Conf. Computer Vision (ACCV)*, 2020. https://openaccess.thecvf.com/content/ACCV2020/html/Tran_GAN-based_Noise_Model_for_Denoising_Real_Images_ACCV_2020_paper.html
- [4] A. Maleky, S. Kousha, M. S. Brown, and M. A. Brubaker, "Noise2NoiseFlow: Realistic Camera Noise Modeling Without Clean Images," *arXiv preprint arXiv:2206.01103*, 2022. <https://arxiv.org/abs/2206.01103>
- [5] J.-Y. Zhu, T. Park, P. Isola, and A. A. Efros, "Unpaired Image-to-Image Translation Using Cycle-Consistent Adversarial Networks," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, 2017, pp. 2223–2232. <https://arxiv.org/abs/1703.10593>
- [6] Y. Choi, M. Choi, M. Kim, J.-W. Ha, S. Kim, and J. Choo, "StarGAN: Unified Generative Adversarial Networks for Multi-Domain Image-to-Image Translation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018, pp. 8789–8797. <https://arxiv.org/abs/1711.09020>